VINUNIVERSITY

Khôi phục sau thảm họa và tính sẵn sàng cao cho hạ tầng AI

AICB-P2T2 · Ngày 23 · Chương 5 — Vận hành

Giảng viên Nguyễn Hải Dương

VinUniversity · Phase 2 · Track 2 · Tuần 5

HÃY SUY NGHĨ...

“Vùng (region) us-east-1 ngừng hoạt động, và điểm cuối phục vụ mô hình (model serving endpoint) của hệ thống nằm tại đó. Sau bao nhiêu phút người dùng nhận ra sự cố, và nhóm vận hành có biết đáp án trước khi sự cố xảy ra không?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội dung bài học

1. RTO và RPO cho Hệ Thống AI
2. Kiến Trúc Nhiều Vùng
3. Khôi Phục Thành Phần Có Trạng Thái
4. Chịu Lỗi Ở Tầng Ứng Dụng LLM
5. Tự Động Hoá Chuyển Đổi Dự Phòng
6. Chi Phí Công Suất Dự Phòng
7. Diễn Tập DR

Mục tiêu bài học

Bài học về khôi phục sau thảm họa (disaster recovery, DR) và tính sẵn sàng cao (high availability, HA) cho hạ tầng AI. Sau buổi học, người học có thể:

1. Xác định thời gian khôi phục mục tiêu (RTO) và điểm khôi phục mục tiêu (RPO) riêng cho từng thành phần của hệ thống AI.
2. Thiết kế kiến trúc nhiều vùng cho dịch vụ suy luận (inference) và chọn chiến lược dự phòng (standby) theo chi phí.
3. Lập kế hoạch sao lưu (backup) và sao chép (replication) cho trạng thái (state): trọng số (weights) mô hình, kho vector (vector store), siêu dữ liệu.
4. Làm cho tầng ứng dụng mô hình ngôn ngữ lớn (LLM) chịu được lỗi của nhà cung cấp mô hình.
5. Viết sổ tay xử lý sự cố (runbook) và chạy một buổi diễn tập có đo RTO thực tế.

Các tầng cần có DR

Ứng dụng LLM
cổng, agent, nhà cung cấp

Phục vụ mô hình
GPU, vLLM, độ trễ khởi động

Trạng thái
trọng số, kho vector, siêu dữ liệu

Mạng và vùng
DNS, nhiều vùng, tài khoản

Sản phẩm cuối ngày

Cần nộp

DR runbook, kết quả buổi diễn tập có đo RTO thực tế, và phân tích đánh đổi chi phí.

- Bảng RTO/RPO cho 5 thành phần: dịch vụ suy luận, kho vector, kho đặc trưng (feature store), kho siêu dữ liệu và phụ thuộc vào nhà cung cấp LLM.
- Đoạn Terraform sao chép liên vùng (cross-region replication) cho trọng số mô hình.
- Cấu hình cổng LLM (LLM gateway) với phương án dự phòng (fallback) sang nhà cung cấp hoặc mô hình thứ hai.
- Sổ tay xử lý sự cố một trang khi vùng chính ngừng hoạt động.
- Kết quả diễn tập: RTO đo được so với RTO mục tiêu, kèm khoảng chênh.

RTO và RPO cho Hệ Thống AI

Vì sao hạ tầng AI ngừng hoạt động khác ứng dụng web thông thường, và cách đo khả năng chịu gián đoạn

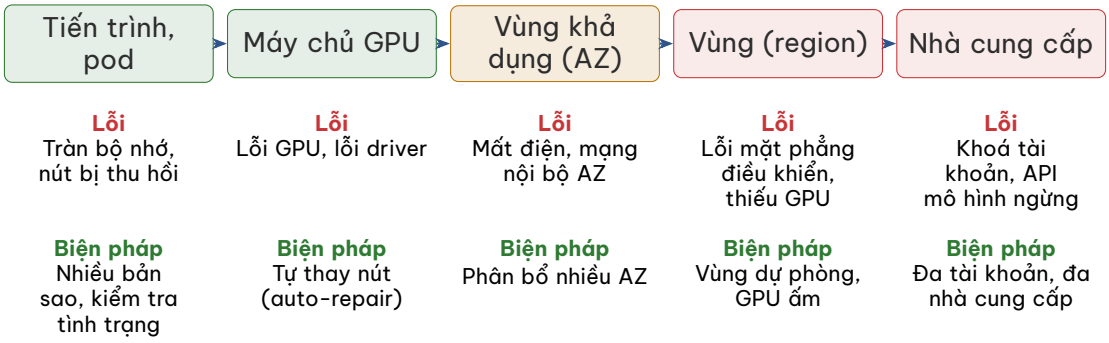
01

Hạ tầng AI khác ứng dụng web thông thường ở đâu

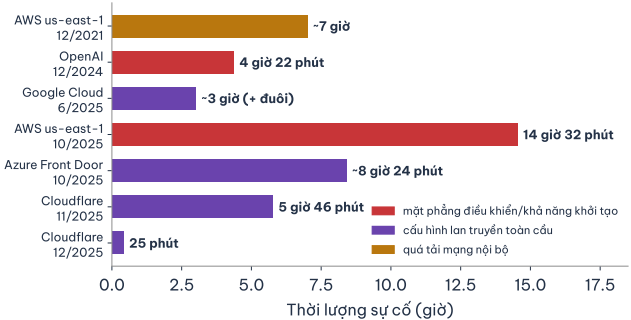
Khía cạnh	Ứng dụng web	Hệ thống AI
Trạng thái cần khôi phục	Bản ghi cơ sở dữ liệu (KB đến GB)	Trọng số mô hình (GB đến TB), kho vector, điểm lưu (checkpoint) huấn luyện
Thời gian khởi động lại	Vài giây	Độ trễ khởi động (cold start) của bộ xử lý đồ hoạ (GPU): tìm công suất, kéo ảnh, nạp trọng số, tổng cộng vài phút
Dữ liệu cần mới	Nhật ký giao dịch	Vector biểu diễn (embedding) và đặc trưng (feature) phải đủ mới
Chi phí dự phòng	Thấp (máy CPU)	Cao: GPU đứng chờ, công suất khan hiếm
Phụ thuộc bên ngoài	Ít	API mô hình của bên thứ ba, hạn mức tần suất theo từng vùng

Số liệu minh hoạ cho từng hàng được kiểm chứng ở các phần sau

Phạm vi sự cố: từ một pod đến cả nhà cung cấp



Sự cố thật: thời lượng và nguyên nhân



Hầu hết sự cố lớn không phải do phần cứng hỏng mà do mặt phẳng điều khiển hoặc cấu hình lan truyền toàn cầu.

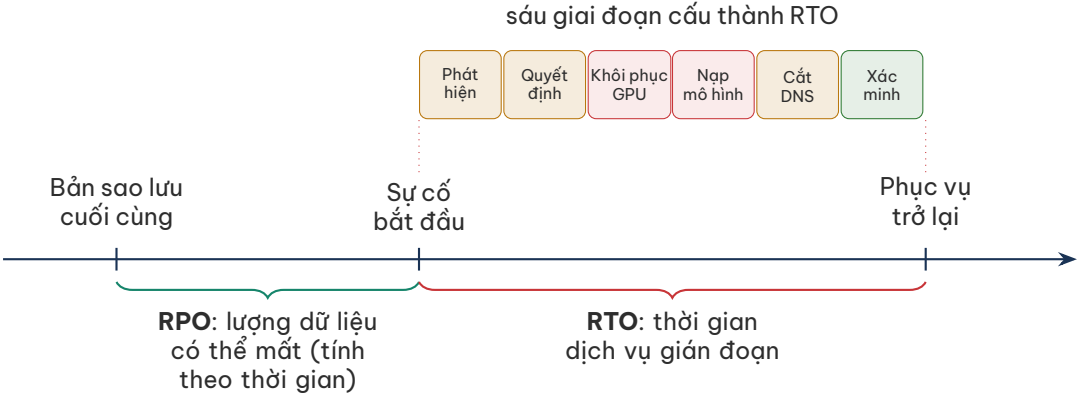
Nguồn: báo cáo sau sự cố của AWS (12/2021, 10/2025), OpenAI (12/2024), Google Cloud (6/2025), Microsoft Azure (10/2025), Cloudflare (11 và 12/2025)

Năm sự cố và bài học cho hạ tầng AI

Sự cố	Điều đã xảy ra	Bài học
AWS us-east-1, 10/2025 (14 giờ 32 phút)	Lỗi DNS của DynamoDB; máy EC2 đang chạy vẫn ổn, việc khởi tạo máy mới và xác thực thất bại	Đường chuyển đổi dự phòng (failover) không được phụ thuộc vào việc khởi tạo máy mới; cấp sẵn công suất
OpenAI, 12/2024 (4 giờ 22 phút)	Dịch vụ giám sát làm quá tải điều khiển Kubernetes; DNS lỗi; việc sửa cần chính mặt phẳng đang hỏng	Có đường vào khẩn cấp (break-glass) nằm ngoài mặt phẳng đang hỏng
Azure Front Door, 10/2025 (8 giờ 24 phút)	Lỗi cấu hình làm sập mặt phẳng dữ liệu toàn cầu; dịch vụ có đường vòng sẵn thì chuyển được	Giữ đường bỏ qua dịch vụ biên toàn cầu (DNS thẳng về nguồn, CDN thứ hai)
Cloudflare, 11/2025 (5 giờ 46 phút)	Tệp cấu hình tự sinh vượt giới hạn làm proxy lỗi; bảng điều khiển cũng ngừng	Coi cấu hình tự sinh là đầu vào không tin cậy; kênh quản trị không nằm chung phạm vi ảnh hưởng
Google Cloud, 6/2025 (3 giờ, kèm đuôi)	Chính sách có trường rỗng sao chép toàn cầu làm sập Service Control; máy khách (client) thử lại (retry) ồ ạt kéo dài phục hồi	Nhiều vùng không cứu được cấu hình nhân bản toàn cầu; thử lại cần nhiễu ngẫu nhiên (jitter)

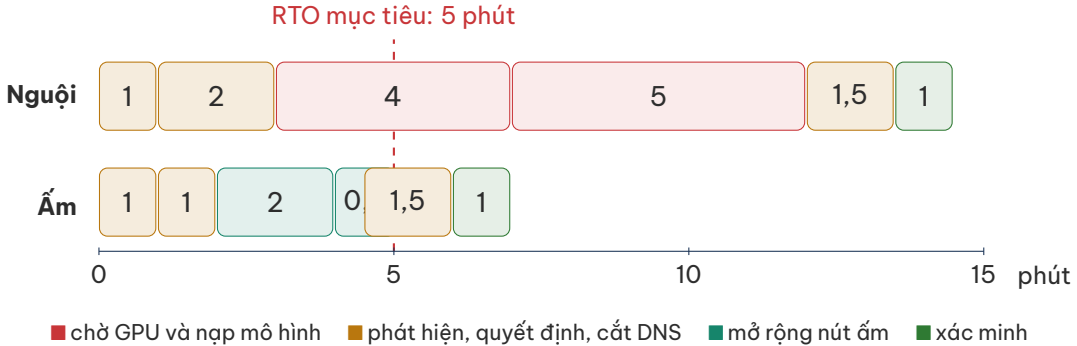
Nguồn: báo cáo sau sự cố (postmortem) do chính nhà cung cấp công bố (đọc 06/10/2026)

RTO và RPO trên một trục thời gian



RPO đo ngược về quá khứ (khoảng cách đến bản sao cuối), RTO đo tiến về tương lai (đến khi phục vụ lại). Mỗi thành phần cần một cặp số riêng.

Vì sao GPU nguội làm RTO vượt mục tiêu



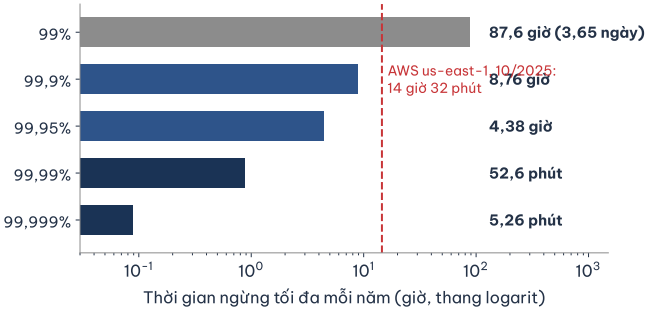
Nút ấm rút RTO từ 14,5 xuống 7 phút, nhưng phát hiện, xác nhận và cắt DNS vẫn chiếm 4 đến 5 phút. Đạt RTO 5 phút cần rút ngắn các bước này hoặc chạy đồng thời hai vùng. Số liệu minh hoạ, đo thật bằng buổi diễn tập.

RTO và RPO theo từng thành phần

Thành phần	RTO	RPO	Lý do
Cổng LLM và API suy luận	5 đến 15 phút	Không áp dụng (không lưu trạng thái)	Người dùng thấy trực tiếp, thoả thuận mức dịch vụ (SLA) chặt
Kho vector	15 phút	5 phút	Cần vector mới, bản sao có thể trễ nhẹ
Kho đặc trưng (feature store)	30 phút	15 phút	Đặc trưng theo lô (batch) chấp nhận độ trễ (latency) lớn hơn
Kho siêu dữ liệu, lịch sử hội thoại	15 phút	Vài giây đến 1 phút	Mất lịch sử làm hỏng trải nghiệm và kiểm toán
Sổ đăng ký mô hình, trọng số	1 giờ	24 giờ	Mô hình ít thay đổi giữa các lần huấn luyện
Nhà cung cấp LLM bên ngoài	Dưới 1 phút (do cổng)	Không áp dụng	Phụ thuộc hạn mức dự phòng, chất lượng mô hình thay thế

Số liệu minh hoạ, điều chỉnh theo SLA thực tế. RTO/RPO càng thấp thì chi phí hạ tầng càng cao; không có một RTO cho mọi thành phần.

Bao nhiêu chữ số 9 là đủ



Ba thành phần nối tiếp, mỗi thành phần 99,9%, cho tính sẵn sàng tổng hợp $0{,}999^3 \approx 99{,}7\%$, tương đương khoảng 26 giờ ngừng mỗi năm.

Đừng thiết kế cho 99,99% nếu SLA chỉ cần 99,9%: chi phí GPU dự phòng tăng phi tuyến.

Kiến Trúc Nhiều Vùng

02

Dự phòng chủ động và thụ động, định tuyến (routing) lưu lượng khi một vùng không còn phản hồi

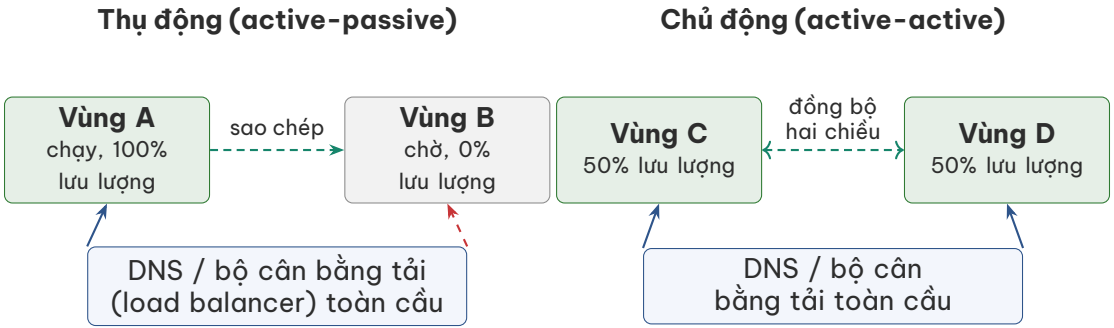
Bốn chiến lược khôi phục sau thảm họa

Sao lưu và khôi phục	Đèn mồi (pilot light)	Dự phòng ấm (warm standby)	Chủ động nhiều vùng
RPO: hàng giờ **RTO: đến 24 giờ**	**RPO: hàng phút** **RTO: chục phút**	**RPO: hàng giây** **RTO: hàng phút**	**RPO: gần 0** **RTO: có thể bằng 0**
Chỉ có dữ liệu sao lưu ở vùng khác	Dữ liệu sao chép sẵn, tính toán tắt cho đến khi cần	Bản thu nhỏ chạy sẵn, mở rộng khi chuyển đổi	Hai vùng cùng phục vụ lưu lượng (active-active)

chi phí và độ phức tạp tăng dần, RTO và RPO giảm dần

Nguồn: AWS Well-Architected Framework, REL13-BP02 (đọc 06/10/2026)

Dự phòng thụ động và chủ động



Rẻ hơn, đơn giản hơn. Chuyển đổi mất vài phút (cắt DNS, khởi động GPU).

RTO gần bằng 0 nhưng chi phí gấp đôi, cần xử lý xung đột trạng thái.

Kiểm tra tình trạng và định tuyến

- Kiểm tra tình trạng (health check) điểm cuối mỗi 10 đến 30 giây.
- 3 lần lỗi liên tiếp thì đánh dấu không khoẻ.
- Bản ghi DNS có TTL thấp (60 giây) để cắt nhanh.
- Định tuyến theo độ trễ: gửi đến vùng gần nhất, loại vùng không khoẻ.

Giới hạn thực tế của DNS

- Bộ nhớ đệm (cache) DNS ở máy khách và nhà cung cấp mạng (ISP) không luôn tôn trọng TTL, một số người dùng vẫn bị ảnh hưởng.
- DNS cộng thêm khoảng 30 đến 90 giây vào RTO.
- Kết hợp bộ cân bằng tải (load balancer) toàn cầu dùng Anycast (như Cloudflare) để cắt nhanh hơn DNS thuần.

Mặt phẳng điều khiển và mặt phẳng dữ liệu

- **Mặt phẳng điều khiển** (control plane): API thay đổi tài nguyên như tạo máy, co giãn, đổi chính sách quản lý danh tính và quyền truy cập (IAM).
- **Mặt phẳng dữ liệu** (data plane): đường xử lý yêu cầu đang chạy.
- Mặt phẳng dữ liệu thường có mục tiêu sẵn sàng cao hơn. AWS khuyến nghị chỉ dùng thao tác của mặt phẳng dữ liệu trong đường chuyển đổi.
- **Ổn định tĩnh** (static stability): loại bỏ phụ thuộc vào mặt phẳng điều khiển khỏi đường khôi phục.

Ví dụ: AWS us-east-1, 10/2025

Máy EC2 đang chạy vẫn khoẻ. Việc khởi tạo máy mới và xác thực thất bại trong nhiều giờ. Một kế hoạch chuyển đổi cần *tạo* GPU mới tại thời điểm sự cố sẽ không chạy được.

Hệ quả thiết kế

Cấp sẵn công suất, không tạo mới trong lúc sự cố.

Nguồn: AWS Whitepaper Disaster Recovery of Workloads on AWS; AWS Fault Isolation Boundaries; AWS Post-Event Summary 10/2025

Suy luận liên vùng không phải là DR

Nền tảng	Tính năng làm gì	Không bảo đảm gì
AWS Bedrock	Hồ sơ suy luận liên vùng (geographic, global) tự chọn vùng xử lý; hồ sơ global rẻ hơn khoảng 10%	Không hỗ trợ Provisioned Throughput; điểm gọi vẫn neo ở vùng nguồn
Microsoft Foundry	Global Standard và Data Zone Standard định tuyến trong phạm vi cho phép	Nếu vùng chính gián đoạn, lưu lượng đã định tuyến vào đó bị ảnh hưởng; không tự chuyển giữa các PTU
Vertex AI	Điểm cuối global của Claude (GA 07/2025) định tuyến đến vùng còn công suất, hạn mức toàn cầu riêng	Chỉ trả theo lượng dùng; Provisioned Throughput chỉ có ở điểm cuối theo vùng

Dùng để tăng thông lượng (throughput) và giảm lỗi 429. Phương án dự phòng thật vẫn nằm ở tầng ứng dụng: cổng LLM, nhiều nhà cung cấp.

Nguồn: tài liệu AWS Bedrock, Microsoft Foundry, Google Cloud (đọc 06/10/2026)

Hạn mức (quota) không phải công suất (capacity)

VINUNIVERSITY

Điều đã được ghi nhận

- Azure: có hạn mức PTU không bảo đảm còn công suất; công suất thay đổi trong ngày theo nhu cầu của mọi khách hàng.
- AWS: kế hoạch đèn mồi có thể không cấp phát được máy khi chuyển đổi, nhất là khi cần đúng loại phiên bản chạy (instance).
- Capacity Blocks của EC2 đặt trước tối đa 8 tuần, tối đa 64 phiên bản chạy mỗi khối, không huỷ được: hợp với huấn luyện hơn là chờ dự phòng.
- Hạn mức GPU của tài khoản mới thường bằng 0 (Ngày 16).

Cách xử lý

- Giữ công suất đặt trước (ODCR) hoặc nút GPU ấm đúng loại phiên bản chạy ở vùng dự phòng.
- Xin nâng hạn mức ở vùng dự phòng trước khi cần.
- Chạy thử cấp phát GPU định kỳ ở vùng dự phòng.
- Biết trước: công suất đặt trước bị tính tiền ngay cả khi chưa dùng.

Chưa có phép đo công khai về nhiều khách hàng cùng chuyển đổi một lúc; đây là suy luận từ thiết kế tài nguyên dùng chung.

Sao chép trọng số liên vùng bằng Terraform

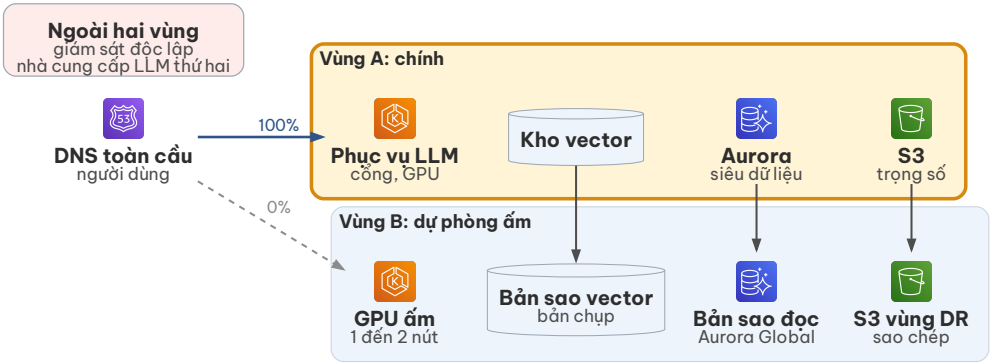
```
resource "aws_s3_bucket_replication_configuration" "weights" {
  bucket = aws_s3_bucket.model_weights_primary.id
  role   = aws_iam_role.replication.arn

  rule {
    id     = "replicate-checkpoints"
    status = "Enabled"
    filter { prefix = "checkpoints/" }
    delete_marker_replication { status = "Disabled" }

    destination {
      bucket        = aws_s3_bucket.model_weights_dr.arn
      storage_class = "STANDARD"
    }
  }
}
```

- Bật versioning ở cả hai bucket, để quay về điểm lưu cũ.
- Sao chép có độ trễ: S3 Replication Time Control (khối `replication_time`, `metrics`) cam kết 99,9% đối tượng trong 15 phút. Không dùng cho RPO dưới 1 phút.
- Kiểm tra checksum sau khi sao chép: mô hình hỏng là lỗi âm thầm.
- Tắt sao chép dấu xoá: xoá nhầm ở nguồn không lan sang DR.
- Chi phí: truyền dữ liệu và lưu trữ nhân đôi; mô hình 70B ở fp16 khoảng 140 GB.

Kiến trúc tham chiếu: ghép các phần lại



Ba lớp cần sao chép riêng: định tuyến (DNS), tính toán (GPU) và trạng thái (S3, Aurora, kho vector). Thiếu một lớp thì chuyển đổi không hoàn chỉnh. Giám sát và nhà cung cấp LLM thứ hai nằm ngoài hai vùng.

Khôi Phục Thành Phần Có Trạng Thái

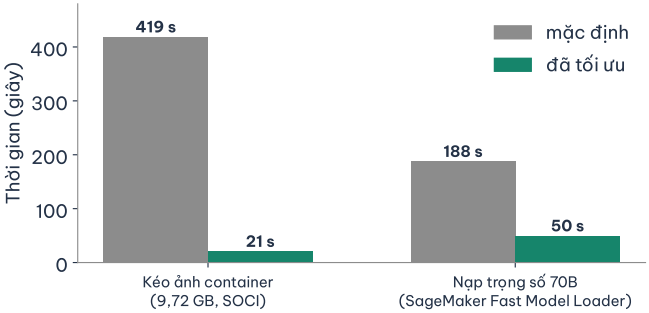
Sao lưu và sao chép cho phần khó nhất: trọng số, kho vector, siêu dữ liệu, điểm lưu huấn luyện

Kiểm kê trạng thái của một hệ thống AI

Trạng thái	Nơi lưu	Tốc độ đổi	Cách bảo vệ
Trọng số, artefact mô hình	Kho đối tượng (object storage)	Chậm	Sao chép liên vùng, versioning
Kho vector	Cơ sở dữ liệu vector	Liên tục	Bản chụp (snapshot), sao chép, dựng lại từ tài liệu gốc
Kho đặc trưng	Offline, online	Theo lô, liên tục	Bản chụp bảng, bản sao
Siêu dữ liệu, sổ đăng ký, lịch sử hội thoại	Postgres	Liên tục	PITR, bản sao liên vùng
Điểm lưu huấn luyện	Kho đối tượng	Mỗi vài chục phút	Điểm lưu bất đồng bộ, bản sao liên vùng
Bộ nhớ đệm KV (KV cache)	Bộ nhớ GPU (VRAM)	Mỗi yêu cầu	Không khôi phục, tính lại
Lời nhắc, cấu hình, tập đánh giá (eval set)	Git	Theo thay đổi	Kho mã ở nhà cung cấp thứ hai

Bản sao mới tính lại bộ nhớ đệm từ lời nhắc

Độ trễ khởi động của một bản sao LLM cỡ 70B



- Kéo ảnh container: từ gần 7 phút xuống 21 giây nhờ nạp lười (lazy loading) bằng SOCI; trọng số vẫn phải nạp sau đó.
- Nạp trọng số 70B: từ 188 giây xuống 50 giây.
- Đo đầu cuối khi mở rộng trên SageMaker vẫn khoảng 306 đến 407 giây (5 đến 7 phút), chưa tính thời gian tìm GPU.

Nguồn: AWS ML Blog (SOCI, 06/2026; Fast Model Loader, 12/2024); các phép đo riêng lẻ, không cộng trực tiếp

Kho vector: cơ chế DR theo từng hệ

Hệ	Cơ chế	Giới hạn cần biết
Pinecone	Bản sao lưu của chỉ mục serverless; khôi phục tạo chỉ mục mới	Chỉ khôi phục cùng project và nhà cung cấp đám mây; chưa hỗ trợ sao chép liên vùng (nhân viên Pinecone, 04/2025)
Qdrant	Sao lưu tăng dần trên đám mây; snapshot theo từng nút khi tự vận hành	Khôi phục vào cụm mới
Weaviate	Sao lưu lên S3, GCS, Azure; khôi phục lên máy khác	Tên và số nút nguồn và đích phải giống nhau
Milvus	Cụm dự phòng bằng CDC, trễ vài giây	Cần Milvus 2.6.6 trở lên; một bản sao CDC
pgvector trên Aurora Global	Sao chép mức lưu trữ, thường trễ dưới 1 giây; đến 10 vùng phụ	Chuyển đổi cần cùng phiên bản engine
OpenSearch	Sao chép liên cụm, thụ động	Sau khi dừng không khởi động lại được; thử với chỉ mục k-NN trước

Nguồn: tài liệu của từng hệ và AWS (đọc 06/10/2026). Tính năng thay đổi nhanh, kiểm tra lại trước khi triển khai.

Điều dễ bị bỏ quên khi khôi phục kho vector

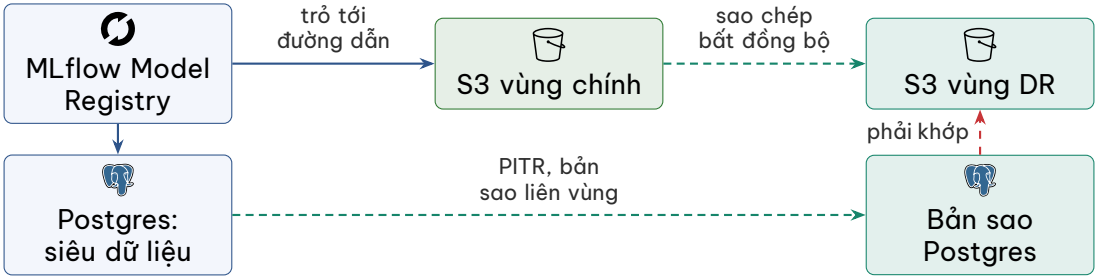
Phiên bản và tính tương thích

- Sao lưu chỉ mục nhưng quên mô hình biểu diễn (embedding model) thì chỉ mục không tương thích khi khôi phục.
- Ghi mã mô hình, phiên bản, số chiều và cách chuẩn hoá vào tệp mô tả (manifest) của bản sao lưu.
- Khôi phục chỉ mục và dịch vụ biểu diễn như một đơn vị.

Dự phòng và kiểm tra

- Dựng lại chỉ mục từ tài liệu gốc luôn khả thi nhưng mất hàng giờ, không phải hàng phút.
- Chỉ mục bị đầu độc dữ liệu (OWASP LLM04) cần khôi phục về thời điểm trước khi bị tác động.
- Bản sao lưu chưa từng khôi phục thử thì chưa được xem là có bản sao lưu.

Sổ đăng ký mô hình và điểm lưu



Nguyên tắc

Siêu dữ liệu của sổ đăng ký và các artefact mô hình phải sao lưu đồng bộ. Sổ đăng ký trỏ đến đường dẫn không tồn tại là lỗi phổ biến nhất khi khôi phục.

Khôi phục về thời điểm (PITR)

- RDS và Aurora: sao lưu liên tục kèm nhật ký giao dịch, khôi phục về bất kỳ giây nào trong tối đa 35 ngày.
- Dùng cho sổ đăng ký tính năng, cơ sở dữ liệu theo dõi thí nghiệm, siêu dữ liệu mô hình.
- Khôi phục tạo một phiên bản chạy mới, không ghi đè phiên bản đang chạy.

Khi PITR chưa đủ

- Khôi phục xảy ra trong cùng vùng: DR thật cần thêm bản sao đọc liên vùng.
- Lỗi logic (migration hỏng) vẫn lan sang bản sao đồng bộ, nên cần bản sao lưu theo thời điểm riêng, không chỉ bản sao.
- Aurora Global Database: độ trễ sao chép thường dưới 1 giây, chuyển vùng phụ lên chính trong chưa đến 1 phút.

Sao lưu bất biến chống mã độc tống tiền và lộ khoá

VINUNIVERSITY

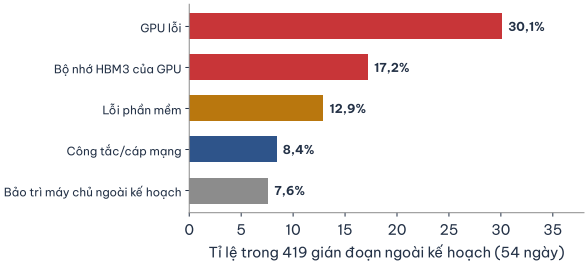
Cơ chế

- S3 Object Lock ở chế độ tuân thủ (compliance): không ai, kể cả người dùng gốc, ghi đè hoặc xoá được trong thời hạn giữ.
- AWS Backup logically air-gapped vault: luôn khoá chế độ tuân thủ, thời hạn giữ tối thiểu 7 ngày, chia sẻ qua RAM để tài khoản khác khôi phục.
- Sao lưu liên tài khoản bảo vệ khỏi tài khoản bị chiếm quyền hoặc mối đe doạ nội bộ.

Vì sao sao chép chưa đủ

- Sao chép chỉ giữ bản sao hiện tại: bản bị mã hoá hoặc bị xoá nhầm cũng được sao chép.
- Cần bản có phiên bản, bất biến và tách tài khoản.
- Sai sót ở vùng chính vẫn có thể lan sang vùng DR nếu cùng tài khoản và cùng khoá.

Điểm lưu huấn luyện: RPO của huấn luyện chính là chu kỳ lưu



- Llama 3 405B: 419 gián đoạn ngoài kế hoạch trong 54 ngày, khoảng một lần mỗi 3 giờ; khoảng 78% do phần cứng.
- Thời gian huấn luyện hiệu dụng vẫn trên 90% nhờ tự động hoá và lưu điểm nhanh.
- PyTorch lưu điểm bất đồng bộ: thời gian chặn huấn luyện giảm từ 148,8 giây xuống 6,3 giây (mô hình 7B).
- RPO huấn luyện xấp xỉ chu kỳ lưu cộng độ trễ ghi bền; điểm lưu chỉ trong RAM mất cùng nút.

Nguồn: Llama 3 Herd of Models (arXiv 2407.21783, mục 3.3.4); PyTorch Blog 11/2024

Chịu Lỗi Ở Tầng Ứng Dụng LLM

Khi nhà cung cấp mô hình gặp sự cố, ứng dụng và agent còn hoạt động được không

Phân loại lỗi của API mô hình ngôn ngữ

Tín hiệu	Ý nghĩa	Hành động
429	Vượt giới hạn tần suất (rate limit) của tổ chức	Chờ theo `Retry-After`; lỗi vượt trần chi tiêu không có `Retry-After`, không thử lại
500, 504	Lỗi nội bộ hoặc quá thời gian chờ	Thử lại có lùi dần, dùng streaming cho yêu cầu dài
529	Nhà cung cấp quá tải (mọi người dùng)	Thử lại ngắn, sau đó chuyển sang phương án dự phòng
200 rồi lỗi trong luồng SSE	Lỗi giữa chừng khi đang sinh	Xử lý trong bộ phân tích luồng, không dựa vào mã trạng thái
200, chất lượng thấp	Suy giảm âm thầm	Chỉ phát hiện bằng bộ đánh giá (eval) và yêu cầu canary

Nguồn: tài liệu lỗi API của Anthropic; hướng dẫn Rate limits của OpenAI (đọc 06/10/2026)

Thử lại đúng cách

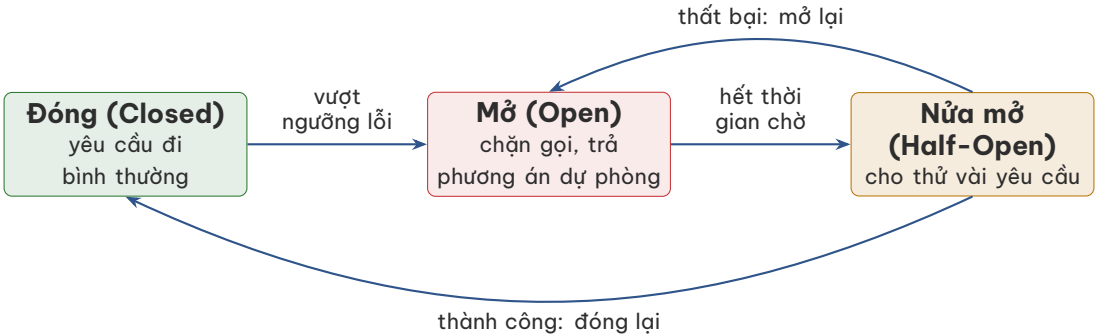
Quy tắc

- Lùi dần theo cấp số nhân kèm nhiễu ngẫu nhiên (exponential backoff with jitter), để các máy khách không thử lại cùng lúc.
- Tôn trọng `Retry-After` khi có.
- Giới hạn số lần thử, rồi chuyển sang phương án dự phòng.
- Yêu cầu thất bại vẫn tính vào giới hạn tần suất của OpenAI: gửi lại liên tục làm tình hình xấu hơn.

Những điều cần tránh

- Thử lại lỗi 4xx về nội dung yêu cầu (400, 401, 403).
- Thử lại không có giới hạn trong lúc nhà cung cấp quá tải.
- Thử lại thao tác có tác dụng phụ (gửi email, ghi cơ sở dữ liệu) khi công cụ (tool) chưa chạy lại an toàn (idempotent).
- Bỏ qua lỗi SSE vì mã trạng thái đã là 200.

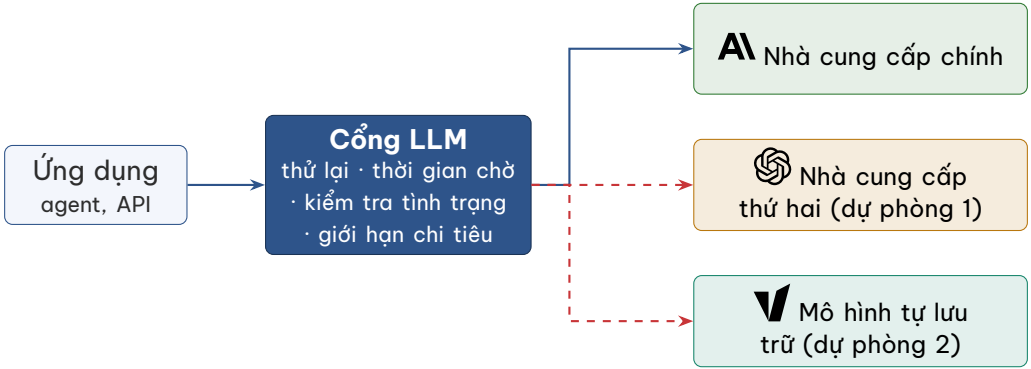
Cầu dao ngắt mạch (circuit breaker)



- Ngăn ứng dụng liên tục gọi dịch vụ chắc chắn đang lỗi; ở trạng thái mở, trả bản sao đã lưu hoặc phương án dự phòng.
- Cổng LLM có cơ chế tương đương: LiteLLM mặc định tạm loại một triển khai khỏi định tuyến trong 5 giây sau 3 lần lỗi (`allowed_fails`, `cooldown_time`).

Nguồn: Azure Architecture Center, Circuit Breaker pattern; tài liệu LiteLLM Routing (06/10/2026)

Cổng LLM (LLM gateway): một điểm vào, nhiều đường ra



Cổng gom chính sách chịu lỗi vào một chỗ: ứng dụng không cần biết nhà cung cấp nào đang phục vụ. Ví dụ: LiteLLM, OpenRouter, Portkey, Cloudflare AI Gateway, Kong, Vercel AI Gateway.

Cấu hình dự phòng trong LiteLLM

```yaml
model_list:
  - model_name: chat-primary
    litellm_params:
      model: anthropic/claude-sonnet-5-5
      api_key: os.environ/ANTHROPIC_API_KEY
  - model_name: chat-backup
    litellm_params:
      model: openai/gpt-5-mini
      api_key: os.environ/OPENAI_API_KEY

router_settings:
  num_retries: 3
  allowed_fails: 3        # default 3
  cooldown_time: 30       # default 5 s
  fallbacks: [{"chat-primary": ["chat-backup"]}]
  enable_pre_call_checks: true
```

Nguồn: docs.litellm.ai, Routing và Reliability (06/10/2026); tên mô hình là ví dụ

- Ứng dụng luôn gọi `chat-primary`; việc chuyển sang `chat-backup` do cổng quyết định.
- Tiêu đề phản hồi `x-litellm-model-id` cho biết triển khai nào đã phục vụ, dùng để đo tỉ lệ dùng dự phòng.
- Cổng cũng là một thành phần cần HA: chạy nhiều bản sao, ở nhiều vùng.

Mô hình dự phòng là một mô hình khác

VINUNIVERSITY

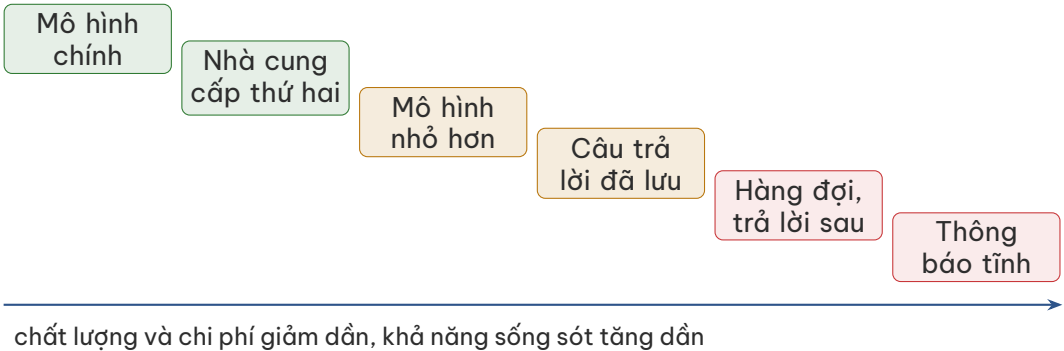
Rủi ro chất lượng

- Lời nhắc (prompt), lược đồ gọi công cụ và đầu ra có cấu trúc hoạt động khác nhau giữa các nhà cung cấp.
- Tham số hợp lệ ở nhà cung cấp này có thể bị từ chối (lỗi 400) ở nhà cung cấp khác.
- Yêu cầu: chạy bộ đánh giá của ứng dụng trên mô hình dự phòng **trước khi** bật chuyển đổi.

Rủi ro chi phí và độ trễ

- Bộ nhớ đệm lời nhắc (prompt cache) không dùng chung giữa các nền tảng; sau khi chuyển, các yêu cầu đầu tiên chạy ở trạng thái lạnh.
- Bộ nhớ đệm ngữ nghĩa (semantic cache) của ứng dụng chỉ còn nếu kho của nó được sao chép sang vùng khác.
- Hạn mức tần suất ở nhà cung cấp dự phòng cần được cấp trước.

Suy giảm có kiểm soát: bậc thang dự phòng



Suy giảm có kiểm soát (graceful degradation): mỗi bậc là một quyết định có chủ đích: người dùng nhận được gì, và ứng dụng thông báo ra sao. Bậc cuối cùng luôn tồn tại, kể cả khi mọi nhà cung cấp ngừng hoạt động.

Agent mất gì khi tiến trình hoặc vùng ngừng hoạt động

VINUNIVERSITY

Bị mất nếu chỉ giữ trong bộ nhớ

- Luồng sinh đang chạy và ngữ cảnh hội thoại.
- Vị trí hiện tại trong vòng lặp của agent.
- Kết quả công cụ chưa được ghi lại.
- Bộ nhớ đệm lời nhắc ở nhà cung cấp cũ.

Thực thi bền vững (durable execution)

Ghi kết quả từng bước vào kho bền vững; sau sự cố, tiếp tục từ bước cuối đã ghi.

Công cụ

- Temporal: tiếp tục từ lịch sử sự kiện
- LangGraph: điểm lưu (checkpoint) vào Postgres
- Cloudflare Workflows: tiếp tục từ bước thành công cuối

Tác vụ (activity) chạy ít nhất một lần, có thể chạy lại một phần. Công cụ có tác dụng phụ cần khoá chống trùng (idempotency key).

Nguồn: docs.temporal.io; docs.langchain.com (LangGraph persistence); developers.cloudflare.com (Workflows). Kho điểm lưu cũng phải được sao chép sang vùng khác.

Mô hình bị ngừng hỗ trợ là một kịch bản DR có lịch

Nhà cung cấp	Chính sách thông báo	Hệ quả khi hết hạn
Anthropic	Tối thiểu 60 ngày trước khi ngừng; Sonnet 4.5 ngừng ngày 30/11/2026	Yêu cầu tới mô hình đã ngừng thất bại
OpenAI	Tối thiểu 6 tháng với mô hình GA, 3 tháng với biến thể chuyên biệt	Mô hình bị tắt theo lịch công bố
Azure (Foundry)	Mô hình GA ngừng sau 18 tháng, không gia hạn	Mọi yêu cầu trả 410 Gone

Cách phòng ngừa: ghim mã mô hình có ngày; lưu lời nhắc và tập đánh giá trong git; chặn triển khai khi điểm đánh giá giảm; giữ tuyến mô hình cũ cách một cờ cấu hình.

Nguồn: trang ngừng hỗ trợ mô hình của Anthropic, OpenAI, Microsoft Foundry (06/10/2026)

Tính sẵn sàng của LLM: đo gì và nhà cung cấp cam kết gì

Bộ SLO đề xuất cho điểm cuối LLM

- Tỉ lệ yêu cầu thành công (không tính lỗi 4xx).
- Thời gian đến token đầu tiên (TTFT), phân vị 95.
- Số token mỗi giây, trung vị.
- Tỉ lệ lỗi giữa luồng sinh.
- Tỉ lệ dùng phương án dự phòng.
- Điểm đánh giá của yêu cầu canary theo chu kỳ.

Chuẩn OpenTelemetry cho GenAI (metric `gen_ai.server.time_to_first_token`) vẫn ở trạng thái Development.

SLA của Amazon Bedrock

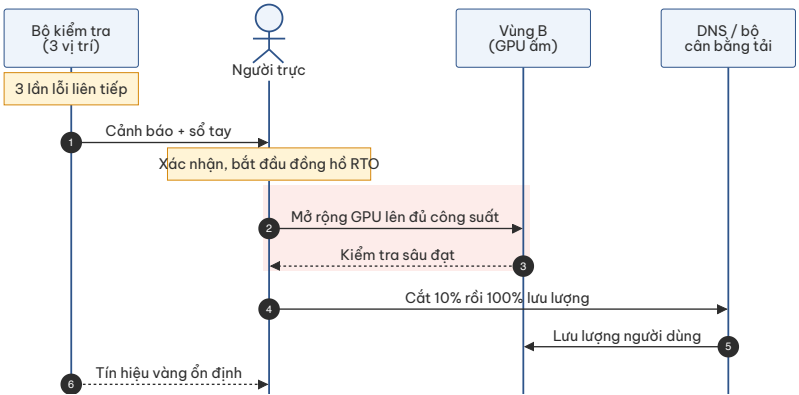
- Hoàn tín dụng dịch vụ 10% khi tính sẵn sàng thấp hơn 99,9%, 25% khi thấp hơn 99,0%, 100% khi thấp hơn 95,0%.
- Chỉ tính yêu cầu trả lỗi 500, theo từng vùng.
- Không cam kết TTFT hay chất lượng đầu ra.

Tín dụng không bù được thiệt hại kinh doanh: SLA của nhà cung cấp không thay thế cho thiết kế chịu lỗi.

Tự Động Hoá Chuyển Đổi Dự Phòng

Từ kiểm tra tình trạng đến cắt lưu lượng, GPU dự phòng, sổ tay xử lý sự cố và chuyển ngược

Trình tự chuyển đổi dự phòng



Lần chuyển đổi đầu tiên nên bán tự động (cảnh báo và một lần xác nhận), không tự động hoàn toàn, để tránh dao động (flapping) giữa hai vùng. Bộ kiểm tra chạy từ 3 vị trí, mỗi 15 giây.

Kiểm tra tình trạng nông và sâu

Kiểm tra nông (shallow)

- Gọi `/health` hoặc `/healthz`, nhận HTTP 200.
- Chỉ cho biết tiến trình còn sống.
- Không phát hiện GPU treo, mô hình chưa nạp xong hoặc kho vector không truy cập được.

Kiểm tra sâu (deep) cho hệ thống AI

- Gửi yêu cầu tổng hợp (synthetic request) sinh 1 token, đo thời gian đến token đầu tiên (TTFT).
- Truy vấn thử một vector cố định từ kho vector.
- Đọc chỉ số GPU (lỗi XID, lỗi ECC) từ DCGM.
- Kiểm tra sâu tốn tài nguyên GPU: chạy thưa hơn, từ nhiều vị trí.

Chống dao động: ngưỡng, thời gian chờ và xác nhận



1. Lỗi liên tiếp từ nhiều vị trí (ngưỡng).
2. Ngưỡng vượt qua: báo người trực.
3. Người trực xác nhận một lần bấm.
4. Sau thời gian chờ (cooldown), vùng chính ổn định.
5. Kiểm tra phục hồi trước khi xác nhận lỗi.
6. Chuyển ngược hoàn tất, trở về bình thường.

Cầu dao ngắt mạch chặn chuyển đổi qua lại khi tín hiệu không ổn định. Các con số là ví dụ.

Làm ấm GPU ở vùng dự phòng

VINUNIVERSITY

Vấn đề: GPU nguội làm RTO vượt mục tiêu

- Tìm và cấp phát GPU: không bảo đảm còn công suất khi cả vùng gặp sự cố.
- Kéo ảnh container: gần 7 phút cho ảnh 9,72 GB nếu không nạp lười.
- Nạp trọng số 70B: hàng chục giây đến vài phút.
- Đo đầu cuối trên SageMaker: khoảng 5 đến 7 phút cho mỗi bản sao mới.

Giải pháp: dự phòng ấm

- Karpenter (Ngày 16) giữ sẵn 1 đến 2 nút GPU ấm ở vùng dự phòng, mở rộng từ 0 đến N khi chuyển đổi.
- Trọng số nạp sẵn vào bộ nhớ đệm cục bộ của nút, không nạp từ S3 lúc chuyển đổi; bộ nạp song song như Run:ai Model Streamer rút ngắn bước này.
- Chế độ ngủ (sleep mode) của vLLM: đánh thức mô hình nhỏ trong 0,26 giây thay vì 37,6 giây; chỉ có tác dụng trong cùng tiến trình.
- Đánh đổi: chi phí nút ấm so với RTO (Phần 6).

Chuyển ngược (failback): khó hơn chuyển đổi

VINUNIVERSITY

Vì sao khó hơn

- Vùng dự phòng đã nhận dữ liệu mới trong lúc sự cố: kho vector, siêu dữ liệu, lịch sử hội thoại.
- Sao chép phải đảo chiều trước khi cắt lưu lượng, nếu không dữ liệu mới bị mất.
- Vùng chính vừa hồi phục thường chưa ổn định.

Quy trình đề xuất

1. Đồng bộ ngược dữ liệu và kiểm tra tính nhất quán.
2. Làm ấm GPU và kiểm tra tình trạng sâu ở vùng chính.
3. Chuyển 5%, 25%, 100% lưu lượng, theo dõi tín hiệu vàng (golden signals) ở mỗi bước.
4. Giữ vùng dự phòng ở trạng thái ấm thêm một khoảng quan sát.

Chuyển ngược nên diễn ra trong giờ thấp điểm, có kế hoạch và có người phụ trách. Đây không phải thao tác tự động.

Sổ tay xử lý sự cố: vùng chính ngừng hoạt động

VINUNIVERSITY

- ☑ **T+0** Xác nhận sự cố: kiểm tra tình trạng từ nhiều vị trí và trang trạng thái của nhà cung cấp đám mây
- ☑ **T+2** Mở kênh sự cố, ghi giờ bắt đầu để tính đồng hồ RTO
- ☑ **T+3** Mở rộng nhóm GPU ở vùng dự phòng từ mức ấm lên đủ công suất
- ☑ **T+6** Kiểm tra trọng số mô hình và độ trễ sao chép của kho vector ở vùng dự phòng
- ☑ **T+8** Cắt 10% lưu lượng sang vùng dự phòng, sau đó 100%
- ☑ **T+12** Xác minh tín hiệu vàng (độ trễ, tỉ lệ lỗi, TTFT) ổn định
- ☐ **Sau sự cố** Đo RTO thực tế, so với mục tiêu, viết báo cáo sau sự cố

Mốc thời gian (phút) là ví dụ; điều chỉnh theo RTO đã cam kết.

Sáu lỗi thường gặp khi thiết kế DR

Sổ tay chưa từng chạy

Sổ tay chỉ nằm trên giấy dễ sai bước khi vận hành lúc áp lực cao.

Cùng tài khoản, cùng khoá

Sự cố IAM hoặc thanh toán làm cả hai vùng ngừng cùng lúc.

Phụ thuộc mặt phẳng điều khiển

Việc chuyển đổi cần gọi API của chính vùng đang hỏng.

Không có cầu dao ngắt mạch

Hai vùng chuyển qua lại liên tục khi kiểm tra tình trạng dao động.

RTO chỉ có trên slide

Chưa có phép đo thật bằng buổi diễn tập.

Giám sát cùng vùng với tải

Bảng theo dõi ngừng cùng lúc, không ai biết hệ thống đang hỏng.

Báo cáo sau sự cố không quy lỗi (blameless postmortem)

VINUNIVERSITY

Mẫu báo cáo

1. Dòng thời gian: bắt đầu, phát hiện, cảnh báo, cắt lưu lượng, khôi phục.
2. RTO và RPO đo được so với mục tiêu, bước nào gây khoảng chênh.
3. Nguyên nhân gốc (5 câu hỏi vì sao), không quy cho cá nhân.
4. Hành động khắc phục có người phụ trách và hạn chót.

Mục riêng cho hệ thống AI

- Số yêu cầu bị từ chối hoặc hết thời gian chờ, tính theo token.
- Chất lượng đầu ra khi chạy bằng mô hình dự phòng.
- Chi phí phát sinh: GPU mở rộng, lưu lượng ra khỏi vùng (egress).
- Khoảng chênh trong sổ tay đã được sửa chưa.

Chi Phí Công Suất Dự Phòng

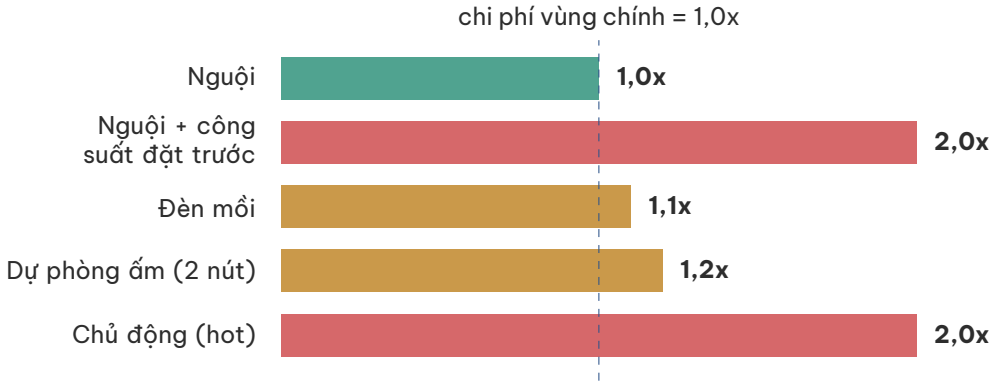
RTO thấp luôn đắt: câu hỏi là đắt bao nhiêu và có đáng không

Nóng, ấm, đèn mồi, nguội

Chiến lược	Mô tả cho suy luận LLM	RTO	Chi phí GPU
Sao lưu và khôi phục	Khởi tạo mọi thứ khi sự cố	Đến 24 giờ	1,0x
Đèn mồi	Giữ siêu dữ liệu, cấu hình, bản sao dữ liệu; mở GPU khi cần	Hàng chục phút	1,1x
Dự phòng ấm	1 đến 2 nút GPU ấm, mở rộng nhanh	Hàng phút	1,2x đến 1,5x
Chủ động (hot)	Đủ công suất ở cả hai vùng	Gần bằng 0	2,0x

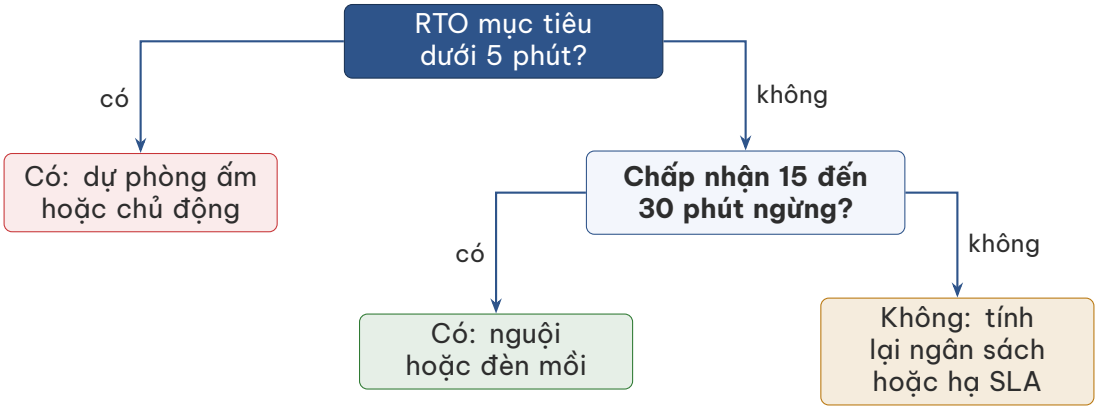
Chi phí GPU là ví dụ minh hoạ, 1x là chi phí GPU của vùng chính; RPO/RTO theo AWS REL13-BP02

Đội 10 nút GPU: chi phí theo chiến lược



Công suất đặt trước được tính tiền như đang chạy, nên mức bảo đảm công suất cao nhất có chi phí gần bằng chạy đồng thời hai vùng. Chưa tính lưu trữ và truyền dữ liệu ra khỏi vùng (egress). Số nút là ví dụ.

Chọn chiến lược nào



Câu hỏi đúng không phải là RTO tốt nhất có thể, mà là RTO nào đủ với chi phí công ty chấp nhận được.

Diễn Tập DR

07

Buổi diễn tập sự cố (game day) là cách duy nhất để biết RTO thật: mô phỏng sự cố và bấm giờ

Vì sao phải diễn tập

Rủi ro khi không diễn tập

- Sổ tay chưa chạy thử chỉ là giả định.
- Sao lưu chưa khôi phục thử có thể không dùng được khi cần.
- Phụ thuộc ẩn (DNS, IAM, hạn mức GPU) chỉ lộ ra khi sự cố xảy ra thật.

Buổi diễn tập (game day) làm gì

- Tạo sự cố có kiểm soát thay vì chờ sự cố thật.
- Đo RTO và RPO thực tế, so với mục tiêu.
- Tìm khoảng chênh trong sổ tay trước khi người dùng gặp phải.
- Huấn luyện nhóm trực xử lý sự cố khi không có áp lực thật.

Game day: quy trình 4 bước



Lên kế hoạch

Chọn kịch bản và phạm vi.
Đặt giả thuyết về RTO.
Chốt điều kiện dừng khẩn cấp.

Thông báo nhóm

Báo nhóm trực và các bên liên quan.
Phân vai: chỉ huy, thực thi, ghi giờ.

Kích hoạt sự cố giả

Chạy ở môi trường thử trước.
Bắt đầu đồng hồ RTO.
Làm theo sổ tay, không đoán.

Đo, rút kinh nghiệm

So RTO đo được với giả thuyết.
Ghi khoảng chênh từng bước.
Cập nhật sổ tay trong 1 tuần.

Kịch bản diễn tập cho hạ tầng AI

Kịch bản	Kiểm chứng điều gì	Mức rủi ro
Xoá một pod phục vụ GPU	Kubernetes tự phục hồi, yêu cầu được thử lại	Thấp
Mất một nút GPU hoặc một AZ	Phân bổ nhiều AZ, thời gian nạp lại mô hình	Thấp đến trung bình
Tăng độ trễ gọi kho vector	Thời gian chờ, phương án dự phòng không có ngữ cảnh	Thấp
Nhà cung cấp LLM trả 429 hoặc 5xx	Cổng LLM chuyển sang mô hình dự phòng	Trung bình
Chặn mạng đến vùng chính	Chuyển DNS, GPU ấm, cắt lưu lượng	Cao
Khôi phục kho vector từ bản chụp	Tính nhất quán, phiên bản mô hình biểu diễn	Trung bình
Thu hồi thông tin xác thực vùng chính	DR không dùng chung tài khoản và khoá	Cao

Công cụ: AWS Fault Injection Service, Azure Chaos Studio, Chaos Mesh, LitmusChaos

Công cụ kỹ thuật hỗn loạn và nguyên tắc an toàn



AWS FIS


Azure Chaos Studio


Chaos Mesh


LitmusChaos


Grafana k6

Nguyên tắc an toàn

- Kỹ thuật hỗn loạn có kiểm soát (chaos engineering): chạy ở môi trường thử trước, sau đó mới đến môi trường thật.
- Có công tắc dừng khẩn cấp (kill switch) ngắt thí nghiệm ngay lập tức. AWS FIS kịch bản liên vùng không có điều kiện dừng sẵn: tự thêm.

Phạm vi

- Báo trước cho người trực, buổi diễn tập không phải bất ngờ.
- Bắt đầu với phạm vi ảnh hưởng nhỏ, mở rộng dần.

Mô hình trưởng thành DR: hệ thống đang ở mức nào

Mức	Tên	Đặc điểm
0	Không có kế hoạch	Sao lưu thủ công, không ai biết RTO thật
1	Sổ tay viết sẵn	Có tài liệu nhưng chưa chạy thử lần nào
2	Chuyển đổi tự động một phần	Kiểm tra tình trạng và cắt DNS, cần người xác nhận
3	Diễn tập định kỳ	Chạy DR drill hàng quý, đo RTO thực tế, cập nhật sổ tay
4	Hỗn loạn có kiểm soát	Tiêm lỗi thường xuyên, chuyển đổi không cần người can thiệp

Mục tiêu thực tế cho hầu hết nhóm: mức 2 đến 3. Mức 4 chỉ đáng đầu tư khi SLA yêu cầu 99,99% trở lên.

Đo RTO bằng mã: thăm dò liên tục trong lúc diễn tập

```python
import json, time, urllib.request
URL = "https://llm.example.com/v1/chat/completions"
HDR = {"Content-Type": "application/json"}
BODY = json.dumps({"model": "m", "max_tokens": 1,
    "messages": [{"role": "user", "content": "ping"}]}).encode()
t_fail = None
while True:
    t = time.time()
    try:
        req = urllib.request.Request(URL, BODY, HDR)
        r = urllib.request.urlopen(req, timeout=3)
        ok, region = r.status == 200, r.headers.get("x-region", "?")
    except Exception:
        ok, region = False, "-"
    if not ok and t_fail is None:
        t_fail = t                # first failure = outage start
    if ok and t_fail and region != "region-a":
        print(f"RTO: {t - t_fail:.0f} s via {region}"); break
    time.sleep(1)
```

- Chạy trước khi chặn lưu lượng, dừng khi vùng dự phòng trả lời lần đầu.
- Yêu cầu 1 token: rẻ nhưng đi qua toàn bộ đường phục vụ.
- Cổng hoặc bộ cân bằng tải trả tiêu đề cho biết vùng phục vụ.
- Đây là RTO nhìn từ người dùng, gồm cả thời gian DNS.

LIVE DEMO

1. **Chuẩn bị**: hai vùng (môi trường thử), dịch vụ suy luận và bản sao kho vector ở cả hai vùng.
2. **Kích hoạt**: chặn lưu lượng đến vùng chính, bắt đầu bấm giờ RTO.
3. **Quan sát**: kiểm tra tình trạng phát hiện lỗi, cảnh báo, cắt DNS.
4. **Xác minh**: yêu cầu mới được vùng dự phòng phục vụ, độ trễ và tỉ lệ lỗi ổn định.
5. **Kết quả**: so RTO đo được với RTO mục tiêu và ghi lại khoảng chênh của từng bước.

LAB #23

Mục tiêu: Thiết kế bảng RTO/RPO cho hệ thống đang xây, viết Terraform sao chép liên vùng cho trọng số, cấu hình cổng LLM có dự phòng, và chạy một buổi diễn tập đo RTO thực tế

Deliverable: Bảng RTO/RPO, đoạn Terraform, cấu hình cổng LLM, sổ tay một trang và kết quả diễn tập (RTO đo được so với mục tiêu)

Thời gian: 2 giờ

Tổng kết

1. RTO và RPO được xác định riêng cho từng thành phần. Hệ thống AI có trạng thái nặng (trọng số, kho vector) và phụ thuộc vào GPU khan hiếm.

2. Chiến lược thụ động đủ cho hầu hết trường hợp. Chủ động chỉ đáng chi phí gấp đôi khi RTO thật sự cần gần bằng 0.

3. Đường chuyển đổi không được phụ thuộc vào mặt phẳng điều khiển của vùng sự cố: cấp sẵn công suất GPU, giữ giám sát ở vùng khác.

4. Tầng ứng dụng LLM cần chịu lỗi riêng: cổng LLM, thử lại có kiểm soát, suy giảm có kiểm soát, thực thi bền vững cho agent.

5. Sổ tay chưa qua buổi diễn tập chỉ là giả định. RTO thật chỉ biết sau khi đo.

Bài tiếp theo

Ngày 24: Quản trị dữ liệu và bảo mật

"RBAC, mã hoá, xử lý thông tin cá nhân (PII), tuân thủ GDPR, ISO 27001 và Nghị định 13: bảo vệ dữ liệu nhạy cảm trong chuỗi xử lý (pipeline) AI."

Bài tập về nhà

- Hoàn thành Lab 23: sổ tay DR và buổi diễn tập chuyển đổi.
- Mang bảng RTO/RPO đã làm vào buổi sau để đối chiếu với yêu cầu quản trị dữ liệu.
- Đọc trước: Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân.

Tài liệu tham khảo

1. AWS, *Well-Architected Reliability Pillar*, *REL13-BP02* và *Disaster Recovery of Workloads on AWS*.
2. AWS, *Post-Event Summary: Amazon DynamoDB Service Disruption in the N. Virginia Region*, 10/2025. `aws.amazon.com/message/101925`
3. OpenAI, *Dec 11, 2024 incident*; Cloudflare, *18 November 2025 outage*; Google Cloud, *12 June 2025 incident*; Microsoft Azure, *Azure Front Door PIR (YKYN-BWZ)*.
4. AWS Bedrock, *Cross-Region inference*; Microsoft Learn, *Foundry deployment types và high availability*; Google Cloud, *Global endpoint for Claude on Vertex AI*.
5. Grattafiori et al., *The Llama 3 Herd of Models*, arXiv 2407.21783. PyTorch Blog, *Reducing Checkpointing Times*, 11/2024.
6. Tài liệu LiteLLM (Routing, Reliability); Anthropic, *API errors*, *Model deprecations*; OpenAI, *Rate limits*, *Deprecations*.
7. Temporal, *Workflow Execution*, *Activity Definition*; LangGraph, *Persistence*.
8. AWS, *S3 Replication Time Control*, *S3 Object Lock*, *AWS Backup logically air-gapped vault*, *AWS FIS Cross-Region scenario*.
9. Ghi chú nghiên cứu: `decks/track2/_research/day23-*.md`.

Hỏi & Đáp

VINUNIVERSITY

Cảm ơn!

AICB-P2T2 · Ngày 23 · Khôi phục sau thảm họa và tính sẵn sàng cao cho hạ tầng AI

Giảng viên Nguyễn Hải Dương · VinUniversity

lms.vinuni.edu.vn · Slide và template trên LMS